CQF Exam Three: Machine Learning in Finance

Introduction

The objective of this exam is to develop a predictive framework to forecast positive market moves (uptrends) utilizing advanced ensemble gradient boosting techniques.

I chose State Street SPDR S&P 500 ETF Trust (SPY) which represents the largest and most actively traded ETFs in the world. It is designed to track performance of the S&P 500 index.

My target variable is labeled as $Y \in \{0, 1\}$, where 1 represents a sufficiently large positive daily market move of SPY to distinguish it from market fluctuations around zero-mean. The threshold for binary classification depends on most recent volatility σ_t (the 20-day rolling standard deviation of daily log returns) multiplied by a scale factor k. Volatility scale k is chosen as a hyperparameter, which in practice can be calibrated/tuned to achieve target performance in a given market regime. chose $k = 0.5$ which corresponds to a probability of about 31% of observing a positive signal if standardized returns were normally distributed (the realized share of positive labels is 32-34%, see the table below). Under the same assumption about 38% of returns fall within $\pm k\sigma_t$ of zero and are treated as market noise, and the remaining 31% are large negative moves; both groups are assigned to the negative class.

$$y_t = \begin{cases} 1, & r_{t+1} > k\sigma_t \\ 0, & r_{t+1} \leq k\sigma_t \end{cases} \qquad r_{t+1} = \ln \frac{C_{t+1}}{C_t}$$

In this set-up we would long the shares of ETF if the model predicts more than 50% chance of the event $r_{t+1} > k\sigma_t$. The probability threshold 50% is also a model hyperparameter and can be selected based on conservatism of the trader. The larger this threshold is, the more conservative a trader might be. Note that the models are fitted with class-balanced sample weights, so the predicted probabilities are shifted towards the positive class and are not calibrated; the 0.5 cutoff then corresponds to a much lower unweighted probability than 50%, which is why the models flag more days as uptrends than the base rate of positive labels.

The choice of initial feature pool is driven by common factors in technical analysis, e.g. past returns, volatilities, RSI, relative position in Bollinger Bands and so on. The full description is provided in Section Q1(b). All features have daily frequency.

The length of historical dataset should be aligned with observed data frequency. My data sample spans period from 2010-01-01 to 2026-10-14 (the download end date) which covers roughly 16.7 years. Prices are unadjusted for dividends, so all returns in this exam are price returns. I split data into train and test sample with 20% test size in chronological order, which means train sample

covers roughly 13.3 years from early 2010 to mid-2023, leaving the remaining 3.3 years for testing. The training period includes Covid-19 and the 2022 sell-off, so the model is trained on both bull and bear markets, whereas the test period is predominantly a bull market, which matters when reading the strategy results below.

Sample	Start	End	Years	Observations	Positive labels, %
Train	2010-02-02	2023-06-06	13.34	3359	32.21
Test	2023-06-07	2026-10-12	3.35	840	33.69

Q1: Feature engineering

In this section I will examine feature selection techniques, feature transformations and selection criteria for each funnel method and finally I will list feature candidates for subsequent modelling exercise.

I focus on three representative feature selection techniques. I combine correlation analysis, Recursive Feature Elimination (RFE) and Random Forest as Filter, Wrapper and Embedded methods.

(a) Feature selection funnel

Each method looks at different aspect of feature engineering.

Filter: correlation analysis

Filter selection methods represent a generic set of methods without model embedding assumption. In other words, we do not model explicitly how features interact to predict the target and do not select them based on their predictive power when looked at together. Instead, we either look at their individual power to predict the target, or multicollinearity, or joint correlations. Our ML classifier is a tree-based gradient boosting algorithm, which captures non-linear relationship between target and predictors and is insensitive to monotonic rescaling of features. As such, univariate relationships between a single feature and the target are less informative than whether a feature is relevant given the presence of others, which is left to the Wrapper and Embedded methods. Collinearity does not hurt the predictions of trees much, but redundant features split the importance among themselves and waste the limited selection budget. Hence, for this exercise I favor a pairwise correlation filter among features (not against the target): of each pair of features with absolute correlation above the threshold, the later column is dropped. Correlation analysis is performed on the training sample after we prune missing data and nonstationary predictors. I set a correlation threshold of 0.9 which is generally not too punitive.

Once features are filtered after correlation analysis, I further prune them based on Wrapper and Embedded methods. I expect that two methods will select different sets of features therefore I allow for wide enough pool in each method by setting maximum number of predictors to 10 and then select top candidates based on overlapping subset from the two broad pools.

Wrapper: Recursive Feature Elimination

Wrapper methods evaluate a specific ML-algorithm to select candidate features. Each method has its own advantages and is expected to generate a unique pool of candidates. Since we allow for a broad pool of candidates before we further narrow it down, the goal is to set a contest with low selection bar in a single widely used ML technique. In this exercise we will stick to RFE.

RFE (here with a logistic regression estimator, removing one feature per step until 10 remain) filters out predictors which have the smallest absolute regression coefficients at each step. This technique requires feature standard scaling before fitting the model because units of changes of predictors should be comparable when we interpret the results; the scaler and the model are fitted on the training sample only. The method looks at different aspect of feature engineering answering the question: "how strongly a predictor changes the model's estimated odds of positive market move, holding other predictors constant?"

Assuming sigmoid function for probability estimation:

$$P(y = 1 \mid X) = \frac{1}{1 + e^{-(\beta_0 + \sum_{j=1}^{K} \beta_j X_j)}}$$

The coefficients are estimated by maximizing the likelihood function (equivalently, its logarithm; scikit-learn's default additionally applies a mild L2 penalty to the coefficients):

$$L = \prod_{i=1}^{N} P(y_i = 1 \mid X)^{y_i} \left(1 - P(y_i = 1 \mid X)\right)^{1-y_i}$$

We obtain logistic regression estimates:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + \cdots + \beta_K X_K$$

Where one unit increase of predictor X_j, i.e. one standard deviation, multiplies the odds of positive signal by:

$$OR_j = e^{\beta_j}$$

Embedded: Random Forest

Unlike Wrapper methods, which repeatedly refit a model on different feature subsets, embedded methods obtain the feature ranking as a by-product of fitting a single model: the importance scores come from the fitted trees themselves. Since in the next chapter I will feed top candidates into gradient boosting algorithms capturing non-linear relationship, Random Forest (300 trees of maximum depth 5, fitted on the training sample) will be a good starting point as a preliminary contest aiming at picking predictors doing well at predicting target together. I keep the 10 features with the highest importance, a high pass rate, as I will further need to intersect candidates with those from the Wrapper method. Impurity-based importance is computed in-sample and tends to favour continuous features with many split points, so it is used here only as a coarse ranking.

Unlike logistic regression, Random Forest does not estimate coefficients β_j. Instead, each decision tree chooses feature splits that reduce node impurity. For classification, I use Gini impurity measure which equals for binary target:

$$G(t) = 2p_t(1 - p_t)$$

The impurity reduction from splitting node t is:

$$\Delta G(t) = G(t) - \frac{n_L}{n_t} G(L) - \frac{n_R}{n_t} G(R)$$

The importance of feature X_j aggregates the weighted impurity reductions across all splits using that feature, averaged across trees and normalized.

$$I_j \propto \sum_{\text{trees } t:} \sum_{\text{split on } j} \frac{n_t}{N} \Delta G(t)$$

A feature with higher importance means it contributes more to distinguish market conditions with positive moves from noise and negative moves, allowing for nonlinear relationships and interactions between predictors.

(b) Feature pool and stationarity

I start with downloading daily time series for Open, Close, High, Low and Volume for the SPY ETF (days with zero volume are removed). Below I focus on common features in technical analysis such as momentum or relative position.

Prices and volume levels are non-stationary, so I build features from returns, fractional changes and bounded oscillators, which are expected to be approximately stationary (this is tested below) and are defined as log returns, fractional price differences or percentage price changes. I form suitable feature pool as illustrated below. I include 5-day, 10-day and 15-day windows and originally have 26 features.

N	Feature	Transformation	Economic interpretation
1	GAP	Gap up or down relative to the previous close.	Overnight repricing between sessions.
2	RET	Daily log return of close.	One-session proportional price movement.
3-5	PCHGX	Close percentage change over the window. Window = X.	Price momentum over the specified lookback.
6-8	VCHGX	Volume percentage change over the window. Window = X.	Change in trading activity over the specified lookback.
9-11	RETX	Rolling sum of daily log returns over the window. Window = X.	Cumulative log price movement over the lookback.
12-14	GAPX	Rolling mean of gaps. Window = X.	Average overnight repricing over the lookback.
15-17	STDX	Rolling standard deviation of daily log returns. Window = X.	Realized return volatility over the lookback.
18-20	BBX	Close position within the Bollinger bands, which use two standard deviations. Window = X.	Relative price position within the Bollinger envelope; values can fall outside 0 to 1.
21-23	RSIX	Relative Strength Index calculated over the window. Window = X.	Momentum oscillator on a 0 to 100 scale.
24-26	STOCHX	Stochastic oscillator over the window with smooth_window=1. Window = X.	Close location within the recent high-low range, on a 0 to 100 scale.

Stationarity is an important property of time series because non-stationary time series might perform well at predicting the target on training data by chance and due to unstable nature exhibit

opposite behavior in a different regime making this relationship spurious. Therefore, I first run the Augmented Dickey–Fuller (ADF) and Kwiatkowski–Phillips–Schmidt–Shin (KPSS) tests and then prune the predictors when they fail at least one of the tests.

The ADF and KPSS tests are used to assess whether a time series is stationary. The tests have opposite null hypotheses.

ADF test:

$$\Delta x_t = \alpha + \gamma x_{t-1} + \varphi \Delta x_{t-1} + \epsilon_t$$

where $\Delta x_t = x_t - x_{t-1}$.

Hypotheses:

- $H_0 : \gamma = 0$ — the series contains a unit root.
- $H_1 : \gamma < 0$ — the series is stationary around a constant.

Decision rule (5% significance):

- p-value < 0.05: Reject the unit-root hypothesis.
- p-value ≥ 0.05: Fail to reject the unit-root hypothesis.

KPSS test:

$$x_t = \alpha + r_t + \epsilon_t$$

$$r_t = r_{t-1} + u_t$$

Hypotheses:

- $H_0 : V(u_t) = 0$ — the series is level-stationary.
- $H_1 : V(u_t) > 0$ — the series contains a stochastic trend.

Decision rule (5% significance):

- p-value < 0.05: Reject the stationarity hypothesis.
- p-value ≥ 0.05: Fail to reject the stationarity hypothesis.

Combined interpretation:

ADF p-value	KPSS p-value	Interpretation
< 0.05	≥ 0.05	Evidence consistent with stationarity
≥ 0.05	< 0.05	Evidence consistent with nonstationarity
< 0.05	< 0.05	Conflicting evidence
≥ 0.05	≥ 0.05	Inconclusive evidence

Conclusion: ADF tests for a unit root, whereas KPSS tests for level stationarity. Using both provides complementary evidence about the stationarity of financial time-series features. One lag in ADF refers to a lagged difference in the regression (a constant but no trend is included, and the

lag is fixed rather than selected by an information criterion); one lag in KPSS refers to the long-run variance estimator (level-stationarity version with a constant).

The table below reports the test results on the training sample. KPSS p-values are interpolated and bounded to the $[0.01, 0.10]$ range. A feature is kept only if it passes both tests.

Feature	ADF statistic	ADF p-value	KPSS statistic	KPSS p-value	Decision
GAP	-40.7561	0.0000	0.0531	0.1000	Keep
RET	-40.9111	0.0000	0.0208	0.1000	Keep
PCHG5	-21.4455	0.0000	0.0567	0.1000	Keep
VCHG5	-28.8821	0.0000	0.0918	0.1000	Keep
RET5	-21.2404	0.0000	0.0639	0.1000	Keep
GAP5	-20.0407	0.0000	0.1491	0.1000	Keep
STD5	-11.1721	0.0000	5.1896	0.0100	Drop
BB5	-28.9405	0.0000	0.1094	0.1000	Keep
RSI5	-18.4649	0.0000	0.3210	0.1000	Keep
STOCH5	-25.7527	0.0000	0.1671	0.1000	Keep
PCHG10	-13.4976	0.0000	0.1139	0.1000	Keep
VCHG10	-22.8097	0.0000	0.1030	0.1000	Keep
RET10	-13.2902	0.0000	0.1284	0.1000	Keep
GAP10	-12.3880	0.0000	0.2851	0.1000	Keep
STD10	-6.5557	0.0000	6.1447	0.0100	Drop
BB10	-21.0116	0.0000	0.1533	0.1000	Keep
RSI10	-13.7706	0.0000	0.5752	0.0249	Drop
STOCH10	-19.0969	0.0000	0.2537	0.1000	Keep
PCHG15	-11.3788	0.0000	0.1629	0.1000	Keep
VCHG15	-21.0007	0.0000	0.2178	0.1000	Keep
RET15	-11.1469	0.0000	0.1814	0.1000	Keep
GAP15	-10.1057	0.0000	0.3971	0.0784	Keep
STD15	-4.9952	0.0000	6.7723	0.0100	Drop
BB15	-17.5573	0.0000	0.2532	0.1000	Keep
RSI15	-11.6760	0.0000	0.8178	0.0100	Drop
STOCH15	-16.2670	0.0000	0.3841	0.0840	Keep

The results of stationarity tests show that the rolling volatility features STD5, STD10 and STD15 give conflicting evidence: the ADF test rejects a unit root, but KPSS rejects level stationarity. This is expected as volatility is highly persistent (volatility clustering, slowly decaying autocorrelation), which is typical of long-memory series that sit between stationary and unit-root processes, and it can look like a slowly moving level over the sample. Since a feature is kept only if it passes both tests, I conservatively drop them. Additionally, RSI10 and RSI15 (but not RSI5) fail the KPSS test as well which may happen with slow-moving oscillators capturing momentum over longer windows.

I subsequently drop them from the candidates list before feeding it into correlation analysis. After stationarity test only 21 features are kept.

(c) Correlation filter and final feature list

Correlation analysis can be performed before or after RFE and Random Forest algorithms. I choose to run it before to eliminate correlated features in training models.

After performing correlation analysis with threshold 0.9 I drop 8 predictors `RET5, STOCH5, RET10, BB10, STOCH10, RET15, BB15, STOCH15` and keep remaining 13 variables.

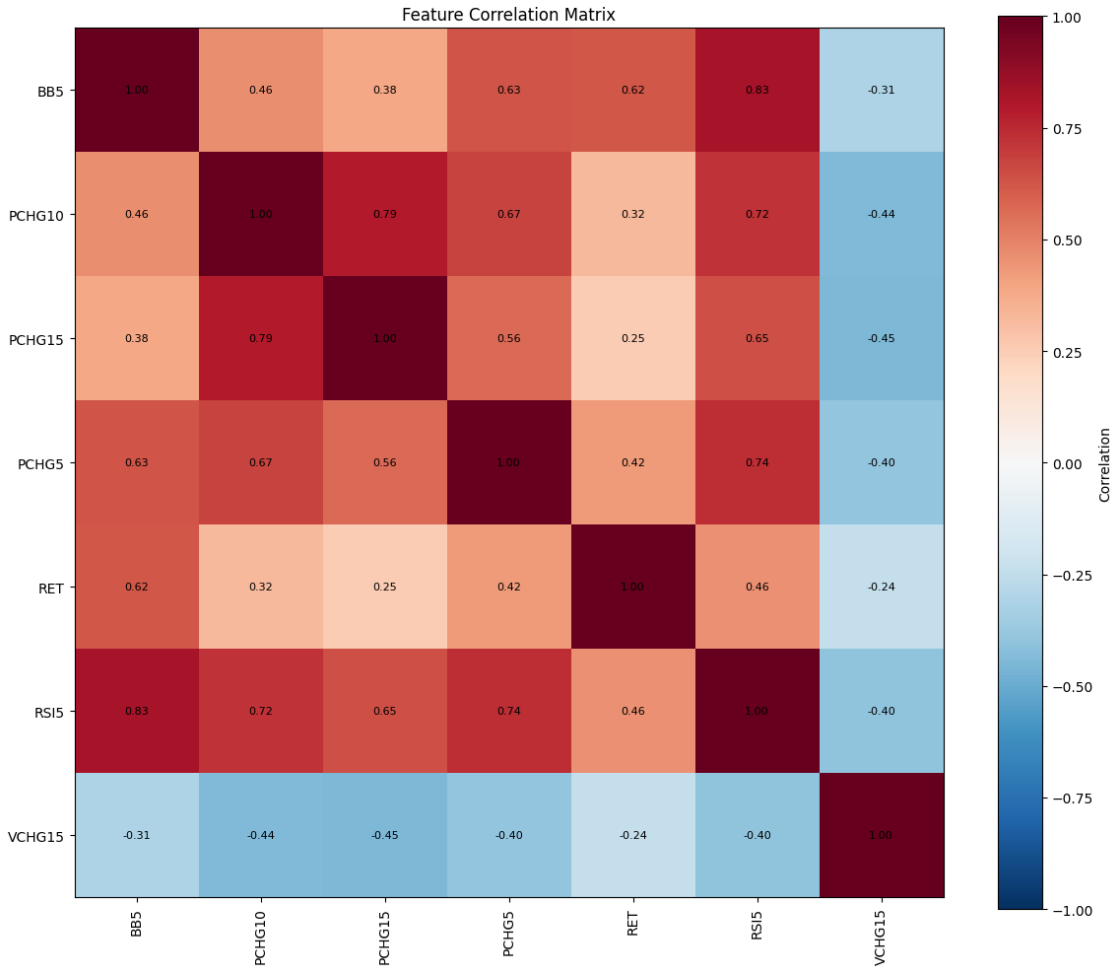
As was mentioned, I set maximum number of features in RFE and RF algorithms to 10 and further select common feature list among their candidates.

#	RFE	random_forest
1	GAP	RET
2	RET	PCHG5
3	PCHG5	VCHG5
4	GAP5	BB5
5	BB5	RSI5
6	RSI5	PCHG10
7	PCHG10	VCHG10
8	GAP10	PCHG15
9	PCHG15	VCHG15
10	VCHG15	GAP15

Final list of features include:

#	Feature
1	BB5
2	PCHG10
3	PCHG15
4	PCHG5
5	RET
6	RSI5
7	VCHG15

Below I demonstrate the correlation heatmap of the final features on the training sample. All pairwise correlations are below the 0.9 threshold. The price-based features (BB5, RSI5, PCHG5, PCHG10, PCHG15, RET) are positively correlated, the strongest pairs being BB5 and RSI5 (0.83) and PCHG10 and PCHG15 (0.79), since they all measure recent price momentum over overlapping windows. The volume change VCHG15 is moderately negatively correlated with them (-0.24 to -0.45): trading volume tends to rise on falling prices, which is common during periods of sell-offs.



We'll now move to next chapter where I discuss gradient boosting algorithms and employ engineered features in this chapter to predict market moves.

Q2: Model Building, Tuning, and Benchmarking

In this chapter I focus on modern Gradient Boosting Decision Tree (GBDT) architectures including XGBoost, LightGBM and CatBoost and assess performance of each modelling framework.

(a) Objective function of GBDT architectures

I'll start with formally defining the mathematical optimization objective function for a GBDT architecture, explicitly highlighting how L1 and L2 regularizations alter the structural tree scores.

Additive model and regularized objective

All three libraries build an additive ensemble of regression trees. Even for classification the trees output raw scores (margins) rather than probabilities. At boosting round t the current margin of observation i is updated by the new tree f_t scaled by the learning rate η:

$$\hat{z}_i^{(t)} = \hat{z}_i^{(t-1)} + \eta f_t(x_i), \qquad f_t(x) = w_{q(x)}, \quad q : \mathbb{R}^d \to \{1, \dots, T\}, \quad w \in \mathbb{R}^T$$

where q assigns an observation to one of the T leaves of the tree and w_j is the score of leaf j. The tree is chosen to minimize the regularized objective

$$\mathcal{L}^{(t)} = \sum_{i=1}^{N} \ell\left(y_i,\ \hat{z}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t)$$

where ℓ is a differentiable loss and Ω penalizes the complexity of the tree.

Notation

$y_i \in \{0, 1\}$ is the label of observation i (1 = uptrend). The margin $\hat{z}_i \in \mathbb{R}$ is the raw ensemble score, i.e. the sum of the (shrunk) outputs of all trees built so far, $\hat{z}_i^{(t-1)} = \hat{z}_i^{(0)} + \eta \sum_{k<t} f_k(x_i)$, where $\hat{z}^{(0)}$ is the initial score. It is the predicted log-odds, $\hat{z}_i = \ln \frac{p_i}{1-p_i}$. The probability of an uptrend is obtained with the sigmoid link

$$p_i = \mathrm{sigm}(\hat{z}_i) = \frac{1}{1 + e^{-\hat{z}_i}}, \qquad \frac{\partial p_i}{\partial \hat{z}_i} = p_i(1 - p_i)$$

and $s_i > 0$ is the sample weight (the pipeline uses balanced sample weights computed from the training labels). The weighted logloss of one observation, as a function of its margin, is

$$\ell_i(\hat{z}_i) = -s_i \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i)\right] = s_i \left[\ln\left(1 + e^{\hat{z}_i}\right) - y_i \hat{z}_i\right]$$

Differentiating with respect to the margin gives the gradient and the Hessian used by all three libraries:

$$g_i = \frac{\partial \ell_i}{\partial \hat{z}_i} = s_i \left(\frac{e^{\hat{z}_i}}{1 + e^{\hat{z}_i}} - y_i\right) = s_i(p_i - y_i), \qquad h_i = \frac{\partial^2 \ell_i}{\partial \hat{z}_i^2} = s_i\, p_i(1 - p_i)$$

both evaluated at $\hat{z}_i = \hat{z}_i^{(t-1)}$.

The gradient g_i gives the local direction in which the margin should move to lower the loss, while the Hessian h_i measures local curvature. It is largest near $p_i = 0.5$ and vanishes as p_i approaches 0 or 1, so confidently classified observations carry little weight in the tree construction.

Second-order approximation and structural tree scores

Regard the per-observation loss as a function of the margin, $\ell_i(z)$, and the new tree output $f_t(x_i)$ as a small step δ away from the current margin $\hat{z}_i^{(t-1)}$ (the learning rate η is applied afterwards as shrinkage). The second-order Taylor expansion $\ell_i(z_0 + \delta) \approx \ell_i(z_0) + \ell_i'(z_0)\,\delta + \frac{1}{2}\ell_i''(z_0)\,\delta^2$ at $z_0 = \hat{z}_i^{(t-1)}$ and $\delta = f_t(x_i)$ gives

$$\ell_i\left(\hat{z}_i^{(t-1)} + f_t(x_i)\right) \approx \ell_i\left(\hat{z}_i^{(t-1)}\right) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2$$

with $g_i = s_i(p_i - y_i)$ and $h_i = s_i p_i(1 - p_i)$ from above. Summing over observations and dropping $\sum_i \ell_i(\hat{z}_i^{(t-1)})$, which does not depend on f_t, gives

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{N} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t)$$

XGBoost defines the complexity penalty with three components:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 + \alpha \sum_{j=1}^{T} |w_j|$$

where γ is a fixed cost per leaf, λ is the L2 penalty and α is the L1 penalty on leaf scores. Let $I_j = \{i : q(x_i) = j\}$ be the observations in leaf j, with aggregated derivatives

$$G_j = \sum_{i \in I_j} g_i, \qquad H_j = \sum_{i \in I_j} h_i$$

Because the tree output is constant within a leaf, the objective separates by leaf:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^{T} \left[G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2 + \alpha |w_j| \right] + \gamma T$$

L2 regularization.

With $\alpha = 0$ for each leaf j the first order condition $G_j + (H_j + \lambda) w_j = 0$ gives the optimal leaf score and the resulting objective:

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \qquad \tilde{\mathcal{L}}^* = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T$$

The score is a Newton step $-G_j/H_j$ multiplied by the shrinkage factor $H_j/(H_j + \lambda)$. The shrinkage is smooth: scores are pulled towards zero but, unlike under L1, do not become exactly zero unless $G_j = 0$ exactly. Each individual gradient $g_i = s_i(p_i - y_i)$ is never zero because $p_i \in (0, 1)$ strictly while $y_i \in \{0, 1\}$; G_j vanishes only if positive and negative g_i cancel within the leaf, which is a non-generic event. The pull is strongest in leaves with small Hessian mass, i.e. leaves supported by few or already confidently classified observations.

L1 regularization.

Unlike L2 regularization, which smoothly shrinks leaf scores towards zero, L1 regularization can set them exactly to zero, effectively disabling leaves with weak predictive signals.

For each leaf j, the regularized objective is

$$\mathcal{L}_j = G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2 + \alpha |w_j|$$

where λ controls L2 regularization and α controls L1 regularization. Minimizing this objective gives the optimal leaf score:

$$w_j^* = -\frac{T_\alpha(G_j)}{H_j + \lambda}, \qquad T_\alpha(G) = \text{sign}(G)\max(|G| - \alpha, 0)$$

The function T_α is the soft-thresholding operator. It reduces the magnitude of the aggregated gradient by α and sets it to zero when $|G_j| \leq \alpha$:

- If $|G_j| > \alpha$: the leaf contributes to predictions, but its score is reduced.
- If $|G_j| \leq \alpha$: the optimal leaf score is zero, so the leaf contributes no correction to predictions.

Substituting the optimal scores into the objective gives the optimal tree-structure score:

$$\tilde{\mathcal{L}}^* = -\frac{1}{2}\sum_{j=1}^{T}\frac{T_\alpha(G_j)^2}{H_j + \lambda} + \gamma T$$

For a candidate split into left and right children, the gain is

$$\text{Gain} = \frac{1}{2}\left[\frac{T_\alpha(G_L)^2}{H_L + \lambda} + \frac{T_\alpha(G_R)^2}{H_R + \lambda} - \frac{T_\alpha(G)^2}{H + \lambda}\right] - \gamma$$

The split is beneficial when the gain is positive and the minimum child requirements are satisfied.

Interpretation: L1 regularization suppresses weak leaf-level signals by setting their scores to zero, whereas L2 regularization continuously shrinks leaf scores. This is particularly relevant for noisy financial returns, where apparent predictive signals may be weak or unstable.

As an illustration, take a leaf with $G = -3$ and $H = 10$, where the leaf improvement is $S = \frac{1}{2}T_\alpha(G)^2/(H + \lambda)$:

λ	α	Leaf score w^*	Leaf improvement S
0	0	0.3000	0.4500
1	0	0.2727	0.4091
5	0	0.2000	0.3000
10	0	0.1500	0.2250

Increasing λ with $\alpha = 0$ lowers the score gradually. Fixing $\lambda = 1$ and increasing α:

λ	α	Leaf score w^*	Leaf improvement S
1	0	0.2727	0.4091
1	1	0.1818	0.1818
1	2	0.0909	0.0455
1	3	0.0000	0.0000

Here $\alpha = 3 = |G|$ removes the leaf completely.

XGBoost

XGBoost uses the objective above without modification. Its penalties are `reg_lambda` (λ, default 1), `reg_alpha` (α, default 0) and `gamma` (γ), while `min_child_weight` is the minimum Hessian sum H required in a child. Trees are grown level by level up to `max_depth`. The search space in `config.cfg` covers `reg_alpha` $\in \{0, 0.01, 0.1, 1, 5\}$, `reg_lambda` $\in \{1, 5, 10, 20, 50\}$ and `gamma` $\in \{0, 0.1, 0.2, 0.3, 0.4\}$.

LightGBM

LightGBM minimizes the same second-order objective with $\Omega(f) = \lambda_1 \sum_j |w_j| + \frac{1}{2}\lambda_2 \sum_j w_j^2$, where λ_1 is `lambda_l1` and λ_2 is `lambda_l2` (both default 0). Its leaf score and split gain have the same form as in XGBoost. LightGBM's Gain is not halved and is compared with `min_gain_to_split` directly, whereas XGBoost compares the same un-halved quantity with γ (the $\frac{1}{2}$ appears only in the objective-reduction derivation):

$$w_j^* = -\frac{T_{\lambda_1}(G_j)}{H_j + \lambda_2}, \qquad \text{Gain} = \frac{T_{\lambda_1}(G_L)^2}{H_L + \lambda_2} + \frac{T_{\lambda_1}(G_R)^2}{H_R + \lambda_2} - \frac{T_{\lambda_1}(G)^2}{H + \lambda_2}$$

There is no explicit γT term. Instead a split is performed only if its gain exceeds `min_gain_to_split` and both children satisfy `min_data_in_leaf` and `min_sum_hessian_in_leaf` (minimum H), while the tree size is capped by `num_leaves`. LightGBM grows the tree leaf-wise, expanding the leaf with the largest gain. Because λ_1 and λ_2 enter the gain, they also change which leaf is expanded next: a leaf with $|G| \le \lambda_1$ has zero gain and is never selected.

CatBoost

CatBoost penalizes leaf scores with L2 only:

$$\Omega(f) = \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

where λ is `l2_leaf_reg` (default 3). There is no L1 term and no per-leaf cost γ, so the soft-thresholding above is not available: leaf scores are always shrunk by the factor $H_j/(H_j + \lambda)$ but never set exactly to zero. CatBoost estimates leaf values with Newton steps (one iteration by default, `leaf_estimation_iterations=1`),

$$w_j = -\frac{G_j}{H_j + \lambda}$$

Trees are symmetric (oblivious): every node of a level uses the same split, so a tree of depth D has $T = 2^D$ leaves and its complexity is controlled by `depth` rather than by γ or a minimum gain. Splits are scored jointly over all leaves of a level using `score_function` (default Cosine). Gradients are computed with the standard (`Plain`) scheme by default together with `has_time=True`, which is discussed in part (b).

(b) Hyperparameter tuning

Consistent search space

One shared grid (defined in `config.cfg`) is mapped to each library's parameter names, so that every model is searched over the same regularization and capacity range wherever the libraries have a comparable parameter. All models are tuned and compared in the same way:

1. The data are split chronologically: the first 80% of days is the training sample and the last 20% the test sample.
2. Hyperparameters are tuned on the training sample only, with a `RandomizedSearchCV` of 100 draws per model, scored by ROC-AUC on a `TimeSeriesSplit` with 5 expanding-window folds and a gap of 1 day. The label of day t uses the close of day $t+1$, so a one-day gap keeps the labels of the training fold from overlapping the validation fold; features built from rolling windows of up to 20 days still overlap across the boundary, so the gap only removes the most direct leakage and the cross-validation scores may be slightly optimistic. Class-balanced sample weights are used when fitting.
3. The best candidate is refitted on the whole training sample and evaluated once on the untouched test sample.

The scoring objective is the same in all three libraries (binary logistic loss for fitting, ROC-AUC for selection), and the random seed is 42 everywhere.

Concept	Shared grid	XGBoost	LightGBM	CatBoost	Rationale
Learning rate	$\{0.01, 0.03, 0.05, 0.07, 0.09\}$	`learning_rate`	`learning_rate`	`learning_rate`	identical shrinkage η
Number of trees	$\{100, 200, 300\}$	`n_estimators`	`n_estimators`	`iterations`	same boosting budget
Tree depth	$\{1, \dots, 5\}$	`max_depth`	`max_depth`	`depth`	a depth-D tree has at most 2^D leaves
Column fraction	$\{0.4, 0.6, 0.8, 1\}$	`colsample_bytree`	`colsample_bytree`	`rsm`	same share of features considered per tree (CatBoost: per split)
Row fraction	$\{0.7, 0.8, 0.9, 1\}$	`subsample`	`subsample`	NA	row sampling; not tuned for CatBoost
Split randomization	$\{0, 0.5, 1, 2\}$	NA	NA	`random_strength`	CatBoost only; noise added to split scores
L2 penalty λ	$\{1, 5, 10, 20, 50\}$	`reg_lambda`	`reg_lambda`	`l2_leaf_reg`	same λ in $H + \lambda$
L1 penalty α	$\{0, 0.01, 0.1, 1, 5\}$	`reg_alpha`	`reg_alpha`	NA	CatBoost has no L1 term

Concept	Shared grid	XGBoost	LightGBM	CatBoost	Rationale
Split cost γ	$\{0, 0.1, 0.2, 0.3, 0.4\}$	`gamma`	`min_split_gain` $= \gamma$	NA	both compare $S_L + S_R - S$ with the threshold (see below)
Minimum Hessian in a child	$\{1, 3, 5, 7\}$	`min_child_weight`	`min_child_weight`	NA	same quantity H_{child}

Parameters that a library does not have are left out rather than approximated, so the CatBoost grid is smaller and the LightGBM grid larger. The scikit-learn names `reg_alpha`, `reg_lambda`, `min_split_gain` and `min_child_weight` used for XGBoost and LightGBM are aliases of the native names `lambda_l1`, `lambda_l2`, `min_gain_to_split` and `min_sum_hessian_in_leaf` used in part (a).

Fixed settings (not tuned)

XGBoost: objective='binary:logistic', tree_method='hist', LightGBM: objective='binary', boosting_type='gbdt', and the library defaults num_leaves=31 and min_child_samples=20, CatBoost: loss_function='Logloss', grow_policy='SymmetricTree', boosting_type='Plain' and has_time=True (see below). The eval_metric/metric entries only name the monitored metric; no validation set or early stopping is used, so the number of trees is chosen by the search, not by early stopping. All libraries use a fixed seed 42 and all available cores (n_jobs=-1, CatBoost thread_count=-1).

Capacity is controlled by depth only

`num_leaves` stays at its default of 31. Since a depth-D tree has at most 2^D leaves, `max_depth` ≤ 4 is the binding cap and `max_depth` $= 5$ is limited marginally by $31 < 32$ leaves, so the three libraries are searched over the same capacity range.

Search coverage

100 draws is an equal computing budget, not equal coverage of the search space. CatBoost's search is the densest as its search grid is narrower, which favours it slightly, and the conclusions below should be read as a comparison of tuned models under an equal budget rather than of fully optimized models.

Leaf-wise (LightGBM) versus level-wise (XGBoost) split selection

Let $S(G, H) = \dfrac{T_{\lambda_1}(G)^2}{H + \lambda_2}$ be the leaf score, with $T_{\lambda_1}(G) = \text{sign}(G)\max(|G| - \lambda_1, 0)$ (for XGBoost $\lambda_1 = \alpha$, $\lambda_2 = \lambda$). For a leaf ℓ with sums (G_ℓ, H_ℓ) and a candidate split s (feature and threshold) into children L and R, the gain is

$$\Delta(\ell, s) = S(G_L, H_L) + S(G_R, H_R) - S(G_\ell, H_\ell),$$

and the reduction of the regularized objective is $\frac{1}{2}\Delta(\ell, s) - \gamma$. Both libraries take the best threshold

inside a leaf, $\Delta^*(\ell) = \max_s \Delta(\ell, s)$, subject to the minimum-Hessian and minimum-data constraints. They differ in which leaf is expanded.

XGBoost, level-wise

Every leaf of the current level $\mathcal{L}_d$ is judged on its own gain:

$$\text{split } \ell \in \mathcal{L}_d \iff \Delta^*(\ell) > 2\gamma \quad (d < \texttt{max_depth}),$$

so $|\mathcal{L}_{d+1}| \le 2|\mathcal{L}_d|$ and the tree has at most 2^D leaves. A leaf with a small gain is still expanded if it clears γ, because the decision does not depend on the other leaves.

LightGBM, leaf-wise (best-first)

At step m the tree has leaf set $\mathcal{L}^{(m)}$ with $|\mathcal{L}^{(m)}| = m + 1$. The leaf expanded next is the one with the largest best gain among the eligible leaves of the whole tree, i.e. those still below the depth cap:

$$\ell^{(m)} = \arg\max_{\ell \in \mathcal{L}^{(m)}_{\text{elig}}} \Delta^*(\ell), \qquad \mathcal{L}^{(m)}_{\text{elig}} = \{\ell \in \mathcal{L}^{(m)} : \ \text{depth}(\ell) < \texttt{max_depth}\},$$

$$\text{stop if } \Delta^*(\ell^{(m)}) \le \texttt{min_gain_to_split}, \ \text{ or } \ |\mathcal{L}^{(m)}| = \texttt{num_leaves}, \ \text{ or } \ \mathcal{L}^{(m)}_{\text{elig}} = \emptyset.$$

Here `max_depth` bounds the depth of each branch and `num_leaves` bounds the total leaf count; whichever binds first ends growth. In this exam `num_leaves` stays at 31, so `max_depth` is the effective cap (see above).

Each step is the greedy choice of the single split that lowers the training loss most. For a budget of M leaves, a level-wise tree spends splits on every leaf of a level, including low-gain ones, whereas leaf-wise growth skips them in favour of the highest-gain leaves. The leaf-wise path therefore reaches a given training-loss reduction with fewer leaves (a greedy statement, not a global-optimality claim). The price is an unbalanced, deeper tree whose depth is not bounded by $\log_2 M$ and which fits noise more easily; this is why `max_depth` and `min_child_samples` are used to cap capacity. Since λ_1, λ_2 enter S, they change $\Delta^*(\ell)$ and hence the ranking of leaves, and a leaf with $|G_\ell| \le \lambda_1$ has zero score and is never chosen.

CatBoost: Target Statistics without look-ahead bias

A naive target statistic for a categorical feature x^i uses all observations, $\hat{x}^i_k = \dfrac{\sum_{j=1}^n \mathbb{1}[x^i_j = x^i_k]\, y_j}{\sum_{j=1}^n \mathbb{1}[x^i_j = x^i_k]}$. It contains y_k itself (and future labels), so $\mathbb{E}[\hat{x}^i_k \mid y_k] \neq \mathbb{E}[\hat{x}^i_k]$: target leakage. CatBoost fixes an ordering σ of the observations and computes the statistic of example k only from the examples that precede it:

$$\hat{x}^i_{\sigma(k)} = \frac{\sum_{j:\, \sigma(j) < \sigma(k)} \mathbb{1}[x^i_j = x^i_k]\, y_j + a\, p}{\sum_{j:\, \sigma(j) < \sigma(k)} \mathbb{1}[x^i_j = x^i_k] + a},$$

where p is a prior (for example the mean of y) and $a > 0$ its weight, which stabilizes categories with few preceding observations. The statistic of k depends only on x_k and on $\{(x_j, y_j) : \sigma(j) < \sigma(k)\}$,

so $\hat{x}^i_{\sigma(k)} \perp y_k \mid x_k$ and the training-time feature has the same conditional distribution as a test-time feature.

With `has_time=True`, σ is the chronological order, $\sigma(j) < \sigma(k) \iff t_j < t_k$, instead of a random permutation. The statistic and the gradient for day t are then built only from days before t, the information set available in real time. All features in this exam are numeric, so there are no categorical Target Statistics to compute and the effect here comes through ordered boosting; the leak-free evaluation still rests on the chronological split and the gapped `TimeSeriesSplit` described above.

Best parameters for xgboost:

subsample	reg_lambda	reg_alpha	n_estimators	min_child_weight	max_depth	learning_rate	gamma	colsample_bytree
0.9	20	0.1	200	5	1	0.01	0.1	1

Best parameters for lightgbm:

subsample	reg_lambda	reg_alpha	n_estimators	min_split_gain	min_child_weight	max_depth	learning_rate	colsample_bytree
0.7	10	0.1	200	0	3	1	0.01	0.8

Best parameters for catboost:

rsm	random_strength	learning_rate	l2_leaf_reg	iterations	depth
0.6	1	0.01	10	200	1

(c) Model metrics and trading strategies

Metric definitions

All metrics are computed out-of-sample on the final 20% of the chronologically ordered data ($N = 840$ days). The positive class $y = 1$ is an uptrend label and the classifier output $\hat{y} = \mathbf{1}[p > 0.5]$ is used for threshold-based metrics. From the confusion matrix

$$\begin{pmatrix} TN & FP \\ FN & TP \end{pmatrix}, \qquad TP + FP + FN + TN = N,$$

we define

$$\text{Accuracy} = \frac{TP + TN}{N}, \qquad \text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall (TPR)} = \frac{TP}{TP + FN},$$

$$\text{Specificity (TNR)} = \frac{TN}{TN + FP}, \qquad \text{FPR} = \frac{FP}{FP + TN}, \qquad F_1 = \frac{2\,\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}},$$

$$\text{Balanced accuracy} = \frac{1}{2}\left(\text{TPR} + \text{TNR}\right).$$

Balanced accuracy is reported because the classes are imbalanced (the base rate of $y = 1$ in the test set is $283/840 = 0.337$), so plain accuracy rewards predicting the majority class. The classification report gives precision, recall and F_1 for each class.

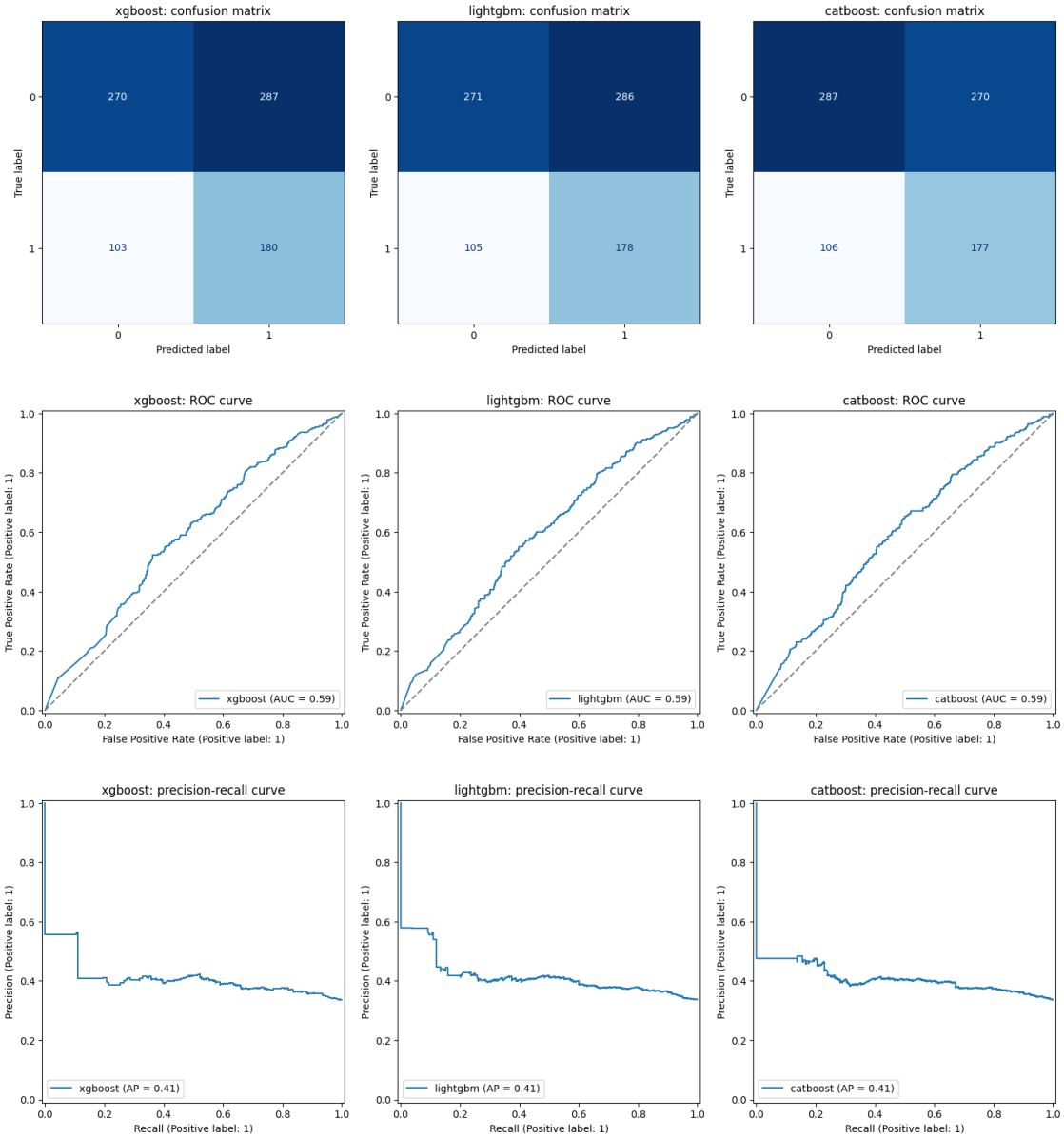
The ROC curve plots $\text{TPR}(\tau)$ against $\text{FPR}(\tau)$ as the threshold τ on p varies, and the precision-recall curve plots $\text{Precision}(\tau)$ against $\text{Recall}(\tau)$. The area under the ROC curve is threshold-free and

equals the probability that a random positive is ranked above a random negative (Mann-Whitney form):

$$\mathrm{AUC} = \int_0^1 \mathrm{TPR}\, d(\mathrm{FPR}) = \frac{1}{n_+ n_-} \sum_{i:\, y_i=1} \sum_{k:\, y_k=0} \left(\mathbf{1}[p_i > p_k] + \tfrac{1}{2}\mathbf{1}[p_i = p_k] \right),$$

with $\mathrm{AUC} = 0.5$ for a random ranking. Overfitting is measured by the train-test gap $\mathrm{BA}_{\mathrm{train}} - \mathrm{BA}_{\mathrm{test}}$ of balanced accuracy. For trading, precision is the hit rate of executed long trades, and the lift over the base rate is $\mathrm{Precision}/\pi_+$ with $\pi_+ = 0.337$.

Fitting results



	xgboost	lightgbm	catboost
train_accuracy	0.5365	0.5448	0.5469
test_accuracy	0.5357	0.5345	0.5524
train_balanced_accuracy	0.5715	0.5762	0.5748
test_balanced_accuracy	0.5604	0.5578	0.5704
roc_auc	0.5862	0.5895	0.5889
f1	0.4800	0.4766	0.4849
precision	0.3854	0.3836	0.3960
recall	0.6360	0.6290	0.6254

Interpretation and recommendation

Predictive power

All three models carry a weak but consistent out-of-sample signal. Test AUC is 0.586 (XGBoost), 0.590 (LightGBM) and 0.590 (CatBoost), against 0.5 for a random ranking, and test balanced accuracy is 0.555-0.570. Plain accuracy (about 0.55 for CatBoost) is below the 0.663 obtained by always predicting the majority class (1 - 0.337), so at the 0.5 threshold the models do not improve overall classification accuracy; this is largely a consequence of the class-balanced weights, which push the models to flag many more days as uptrends than the base rate. AUC and balanced accuracy, which do not depend on the class mix, provide evidence of a weak ranking signal. The standard error of an AUC estimated on 840 observations is roughly 0.02, so AUC of 0.59 is distinguishable from 0.5 only if the daily observations were independent, which they are not (overlapping rolling-window features), and the evidence should be treated as weak. The three algorithms are almost indistinguishable: the differences in AUC (0.004) and balanced accuracy (0.01) are far smaller than the sampling noise on 840 daily observations. The tuned parameters are also at the edge of the search grid for all three libraries (max_depth/depth = 1, learning_rate = 0.01), i.e. the search chose the most heavily regularised, nearly additive models of decision stumps, which is consistent with a very low signal-to-noise ratio.

Precision-recall trade-off

At the 0.5 threshold the models are recall-oriented: recall is 0.62-0.64 but precision is only 0.38-0.40, so roughly six in ten long signals are false positives. Precision is about 1.18 times the base rate of 0.336 (0.396/0.336), a modest lift. The models flag too many days as uptrends (time in market is about 53-56%) and do not separate strong up-days from noise sharply. Raising the probability threshold would increase precision at the cost of recall, and the best threshold depends on trading costs and risk appetite; the threshold should be chosen on validation data, not on the test sample.

Stability and risk

The train-test balanced accuracy gap is small: 0.001 (XGBoost), 0.01 (LightGBM) and 0.006 (CatBoost), so overfitting is limited and the tuned, regularised models generalise. CatBoost has the smallest gap and the best test balanced accuracy, F1, but XGBoost has the best recall. As long/flat strategies, however, all three underperform Buy-and-Hold out-of-sample (Sharpe 0.59-0.72 against 1.22). The paired block bootstrap shows the Sharpe shortfall is significant, with the 95% confidence interval for the difference excluding zero for every model. Position changes (about 180) also reduce Sharpe further once a 1bp cost is applied. The strategies have a slightly lower volatility

and maximum drawdown, but this does not compensate for the lost return during a strong bull market, so the comparison reflects a single, favourable regime for Buy-and-Hold.

Recommendation

CatBoost is the preferred candidate: it has the highest test balanced accuracy, F1, the smallest train–test balanced-accuracy gap, and the strongest strategy Sharpe of the three. However, the models' performance differences are small, and none of the tested long/flat strategies outperformed Buy-and-Hold; therefore, CatBoost should not be deployed as a stand-alone timing strategy at the 0.5 cutoff. Its modestly above-random AUC suggests potential value as an input to a broader strategy, such as gradual position sizing, but first verify probability calibration and develop the sizing rule using validation data, then assess it with walk-forward tests and realistic trading costs. As a further research direction, test categorical features that identify bearish market conditions, ensuring they use only information available at the time of each prediction.

Trading strategy assessment

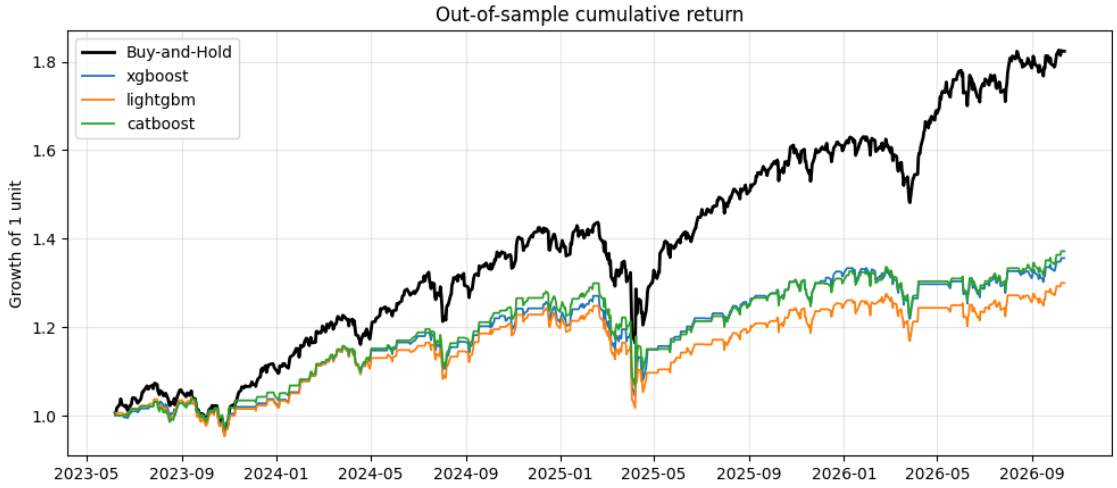
Each model is converted into a long/flat strategy: the position on day t is $s_t = \hat{y}_t \in \{0, 1\}$ and the strategy log return is $r^S_{t+1} = s_t\, r_{t+1}$, where $r_{t+1} = \ln(P_{t+1}/P_t)$ is the SPY log return. The benchmark is Buy-and-Hold, $s_t \equiv 1$. Returns are price returns of unadjusted SPY closes (dividends are excluded, which understates the absolute returns of all strategies, mostly the Buy-and-Hold benchmark), the risk-free rate is set to zero in the Sharpe and Sortino ratios, and, unless stated, transaction costs are ignored; where a cost is applied it is 1bp of return per unit of position change. The paired block bootstrap below resamples blocks of 20 consecutive days (2000 resamples, same blocks for strategy and benchmark) to preserve short-term serial dependence. With $\bar{r}$ and σ the mean and standard deviation of daily returns and 252 trading days per year:

$$\text{Sharpe} = \sqrt{252}\,\frac{\bar{r}}{\sigma}, \qquad \text{Sortino} = \sqrt{252}\,\frac{\bar{r}}{\sigma_d}, \quad \sigma_d^2 = \frac{1}{n}\sum_t \min(r_t, 0)^2,$$

$$\text{MaxDD} = \min_t \left(C_t - \max_{u \le t} C_u \right), \quad C_t = \sum_{k \le t} r_k.$$

Cumulative return C_T and drawdown are therefore expressed in log points, consistent with the `Total_Return` and `Max_Drawdown` figures computed by the pipeline.

	Buy-and-Hold	xgboost	lightgbm	catboost
Cumulative log return	0.6010	0.3046	0.2624	0.3160
Annualised return	0.1803	0.0914	0.0787	0.0948
Annualised volatility	0.1483	0.1338	0.1335	0.1325
Sharpe	1.2160	0.6830	0.5897	0.7155
Sortino	1.7858	0.9928	0.8515	1.0394
Max drawdown	-0.2107	-0.1969	-0.2046	-0.1969
Time in market	1.0000	0.5560	0.5524	0.5321
Position changes	0.0000	180.0000	187.0000	180.0000
Sharpe (1bp cost)	1.2160	0.6426	0.5477	0.6747



Paired block bootstrap of Sharpe(strategy) minus Sharpe(Buy-and-Hold):

	xgboost	lightgbm	catboost
Sharpe diff vs B&H	-0.533	-0.626	-0.500
95% CI low	-1.005	-1.139	-1.003
95% CI high	-0.071	-0.152	0.017
P(diff $>$ 0)	0.007	0.004	0.030

Conclusion

The gradient-boosting models show modest out-of-sample discrimination: AUC is about 0.59 and balanced accuracy is 0.56–0.57. CatBoost is marginally strongest on several metrics, but the three models perform similarly, and the results do not establish a meaningful winner. The tuning grid is limited relative to the number of hyperparameters; broader tuning could be explored, subject to computational cost and a validation design that avoids overfitting.

This modest classification signal did not translate into stronger performance under the tested long/flat rules at a 0.5 cutoff. Their cumulative log returns were 0.26–0.32 versus 0.60 for Buy-and-Hold, and their Sharpe ratios were 0.59–0.72 versus 1.22. The paired block-bootstrap intervals for the Sharpe differences were below zero, indicating underperformance relative to Buy-and-Hold in this test period. Accordingly, these results do not support using the models as stand-alone timing strategies or as a replacement for passive SPY exposure. Their scores could be investigated as inputs to a broader strategy, but calibration, position sizing, additional features and any resulting trading rule would need to be validated out of sample with realistic costs.

Appendix

The features are computed as follows:

$$GAP_t = \frac{O_t}{C_{t-1}} - 1$$

$$RET_t = \ln \frac{C_t}{C_{t-1}}$$

$$PCHG_{X,t} = \frac{C_t}{C_{t-X}} - 1$$

$$VCHG_{X,t} = \frac{V_t}{V_{t-X}} - 1$$

$$RET_{X,t} = \sum_{i=0}^{X-1} RET_{t-i} = \ln \frac{C_t}{C_{t-X}}$$

$$GAP_{X,t} = \frac{1}{X} \sum_{i=0}^{X-1} GAP_{t-i}$$

$$STD_{X,t} = \sqrt{\frac{1}{X-1} \sum_{i=0}^{X-1} \left(RET_{t-i} - \overline{RET}_{X,t}\right)^2}$$

$$BB_{X,t} = \frac{C_t - LB_{X,t}}{UB_{X,t} - LB_{X,t}}, \quad LB_{X,t} = SMA_X(C)_t - 2\sigma_X(C)_t, \quad UB_{X,t} = SMA_X(C)_t + 2\sigma_X(C)_t$$

$$RSI_{X,t} = 100 - \frac{100}{1 + RS_{X,t}}, \quad RS_{X,t} = \frac{AvgGain_{X,t}}{AvgLoss_{X,t}}$$

$$STOCH_{X,t} = 100\,\frac{C_t - Lmin_{X,t}}{Hmax_{X,t} - Lmin_{X,t}}, \quad Hmax_{X,t} = \max_{i<X} H_{t-i}, \quad Lmin_{X,t} = \min_{i<X} L_{t-i}$$

RSI inputs are the gains and losses of the close:

$$Gain_t = \max(C_t - C_{t-1}, 0), \qquad Loss_t = \max(C_{t-1} - C_t, 0)$$

$$AvgGain_{X,t} = \frac{(X-1)\,AvgGain_{X,t-1} + Gain_t}{X}, \qquad AvgLoss_{X,t} = \frac{(X-1)\,AvgLoss_{X,t-1} + Loss_t}{X}$$

Notation: O_t = open; C_t = close; H_t = high; L_t = low; V_t = volume; X = lookback window (trading days). $\sum$ is a sum over the X-day window; SMA_X is its simple moving average. All return-type features (GAP, PCHG, VCHG, RET) are expressed as decimals, not percentages.

Notes: STDX uses sample standard deviation (ddof=1); BBX bands use price standard deviation, not return volatility. RSI uses the chosen gain/loss smoothing convention (e.g., Wilder). Guard against zero denominators and use consistent split-adjusted OHLC prices.

RSI note: AvgGain and AvgLoss use Wilder-style exponential smoothing ($\alpha = 1/X$), initialized from the first X price changes. The stochastic oscillator parameter smooth_window=1 means no smoothing of %K; it does not change the RSI formula.